JOURNAL OF COMPLEXITY 3, 406–428 (1987)

Bin Packing with Divisible Item Sizes

E. G. COFFMAN, JR., M. R. GAREY, AND D. S. JOHNSON

AT&T Bell Laboratories, Murray Hill, New Jersey 07974

We study a variety of NP-hard bin packing problems under a divisibility constraint that generalizes the often encountered situation in which all item sizes are powers of 2. For ordinary one-dimensional bin packing, we show that First Fit Decreasing produces optimal packings under this restriction, and that if in addition the largest item size divides the bin capacity, then even the less powerful First Fit algorithm is optimal. Similar results are obtained for two-dimensional bin packing and multiprocessor scheduling, along with several other simple variants. For more complicated problems, like vector packing and dynamic bin packing, the improvement is less substantial, and we indicate why.

1. INTRODUCTION

The performance of approximation algorithms for bin packing has been studied extensively, both for the classical problem and for a wide variety of its variants. Indeed, this topic has received a certain amount of notoriety for the substantial length and difficulty often required of many of the proofs. In this paper, we consider a class of bin packing problems that is of interest both from the point of view of applications and from the fact that much simpler proofs of performance bounds are possible.

This class of bin packing problems is characterized by the special restriction that the item sizes form a *divisible sequence*, i.e., the sequence of distinct sizes

$$s_1 > s_2 > \cdots > s_i > s_{i+1} > \cdots$$

taken on by the items (the number of items of each size is arbitrary) is such that for all $i \geq 1$, s_{i+1} exactly divides s_i.

We say that a list L of items *has divisible item sizes* if the sizes of the

0885-064X/87 $3.00

items in L form a divisible sequence. Also, if L is a list of items and C is a bin capacity, we say that the pair (L, C) is *weakly divisible* if L has divisible item sizes and *strongly divisible* if in addition the largest item size s_1 in L exactly divides the bin capacity C.

Divisible item sizes are of interest because they arise naturally in certain applications, such as memory allocation in computer systems, where device capacities and block sizes are commonly restricted to powers of 2 (e.g., see the text by Knuth, 1973), and because (as shown in later sections) approximation algorithms for many types of NP-hard bin packing problems produce substantially better packings for such sets of items. In fact, this paper emphasizes those cases in which such restrictions lead to efficient algorithms that are asymptotically optimal, distinguishing them from cases in which, despite the additional restrictions, the performance of standard approximation algorithms is not much different from the general case.

In the classical one-dimensional bin packing problem of Section 2, a given list of items $L = I_1, I_2, \ldots, I_n$ is to be partitioned into a minimum number of subsets such that the items in each subset sum to no more than C. (Throughout the paper, I_j denotes both the name and size of the jth item.) In the standard physical interpretation, we view the items of a subset as having been packed in a bin of capacity C.

Suppose that L and the number m of available bins are taken as parameters, and the objective is to find the smallest C such that L can be packed into m bins of capacity C. We then have the classical multiprocessor (makespan) scheduling problem studied in Section 3. The problem of Section 4 takes L and both m and C as parameters; the objective is to find a largest (in cardinality) subset of L which can be packed into m bins of capacity C. Section 5 deals with our final variant of the basic problem; the goal is a *maximum* value of m such that L can be packed into m bins with the items of each summing to *at least* C. For obvious reasons, this has been called the dual bin packing problem.

Sections 6–8 generalize the bin packing and scheduling problems of Sections 1 and 2 to higher dimensions. Geometric packings in two dimensions are considered in Section 6. Items are given as vectors in two dimensions representing the widths and heights of rectangles. In the strip packing version, items (rectangles) are to be packed in a single "bin" consisting of a vertical strip of given width, so as to minimize the required height of the strip. In the bin packing version, the items are to be packed within a smallest set of identical rectangular regions (two-dimensional bins) of given heights and widths. In both versions items cannot overlap each other or the edges of the enclosing strip or bins, and the sides of the items must be parallel to the sides of the enclosing strip or bins.

Section 7 considers a nongeometric version of vector packing in which items are taken as d-dimensional vectors, as are the bin capacities. The items are to be packed so that the sums in each component do not exceed the bin capacity specified for that component, and the least number of bins is used.

Finally, the dynamic bin packing problem of Section 8 introduces the dimension of time to the problem of Section 2 by allowing items to arrive and depart according to arbitrary processes. Along with C, a problem instance consists of a list of arrival times, departure times, and sizes of n items; the objective is an algorithm that minimizes the maximum number of bins ever required by currently active items during the time span defined by the problem instance.

In the one-dimensional problems of Sections 2–5, complexity depends primarily on the set, S_I, of possible item sizes, and except for Section 3, the set of possible capacities. Intuitively, divisibility is a very strong restriction on S_I. A little reflection leads one to expect major reductions in the complexity of these packing problems, especially when C is a multiple of all item sizes. And indeed, we show in each case that certain simple approximation algorithms for the general case become optimal for strongly divisible instances.

On the other hand, each of the generalizations in Sections 6–8 introduces additional structure which complicates the packing problem in fundamental ways. It is much less clear how far constraints applied only to item sizes will simplify these problems, and indeed they offer major help only in the two-dimensional geometric packing problems of Section 6.

In addition to the generalizations of Sections 6–8 there are many others not considered here. The set of item sizes can even be trivialized (all item sizes are taken to be equal), and NP-hard problems remain. A classic example is the multiprocessor scheduling problem where the item sequences (schedules) in the m bins must satisfy the precedence constraints of a partial order taken as part of the problem instance. (See Ullman, 1975; Garey and Johnson, 1979.) Another example is the scheduling of messages (items) in a computer/communication network (Coffman *et al.*, 1985).

There are also, of course, further simple variants on the basic problem for which the efficacy of the divisibility constraint remains unexamined. Some of these will be mentioned in passing and others will no doubt occur to the reader. We confine ourselves here to just a few of the more fundamental packing variants. We also leave as open problems the effects of restrictions other than divisibility on the S_I, for instance the restriction to Fibonacci numbers (as suggested in the context of memory allocation, for example, by Hirschberg (1973)).

2. Standard One-Dimensional Bin Packing

In the standard one-dimensional bin packing problem, we are given a capacity C and a list of items $L = I_1, I_2, \ldots, I_n$, and are asked to partition the items into a minimum number of subsets such that the items in each subset sum to no more than C. We normally picture the items in a subset as being contained in a "bin," hence the name of the problem. In describing packing algorithms, we assume an unbounded sequence of initially empty bins $B_1, B_2, \ldots$, and specify how the items are assigned to bins. According to the *First Fit* (*FF*) algorithm, items are packed in order and the next item to be packed is assigned to the lowest-indexed bin having an unused capacity no less than the size of the item; i.e., if I_j is packed in B_k, then k is the least index such that the subset of $I_1, \ldots, I_{j-1}$ already assigned to B_k sums to no more than $C - I_j$.

Note that FF is on-line with respect to the list L, in that it does not use information about items that follow the current item in L. If the FF algorithm is preceded by an initial sorting of L into decreasing order, then we have the off-line *First Fit Decreasing* (*FFD*) algorithm. First Fit Increasing (FFI) is similarly defined by an initial sorting of L into increasing order.

Next Fit (*NF*) is an on-line algorithm considered in later sections. According to NF the first item is packed in B_1; thereafter the next item to be packed is placed in the highest-indexed nonempty bin, if it fits; otherwise, it starts a new bin ($B_1, \ldots, B_j$ are considered full once B_{j+1} receives its first item). NFD and NFI are defined as for FF.

In an analysis of our class of bin packing problems, the following two easily proved facts are quite useful.

Fact 1. *If the pair* (L, C) *is strongly divisible, then the gap (remaining available space) in any partially filled bin will always be an integer multiple (possibly zero) of the size of the smallest item in the bin.*

Fact 1 is not true in general if (L, C) is only weakly divisible.

Fact 2. *If* L *has divisible item sizes, then any set of items from* L *that individually do not exceed* s_i *and that in total sum to at least* s_i *(where* s_i *is one of the sizes in the divisible sequence) must contain a subset that sums exactly to* s_i.

Johnson *et al.* (1974) showed that, in the general case, FF and FFD have the tight asymptotic performance bounds of $17/10 \cdot \mathrm{OPT}(L)$ and $11/9 \cdot \mathrm{OPT}(L)$, respectively. By contrast, the following theorems show that FFD is optimal for weakly (and hence strongly) divisible instances (L, C), and that FF is optimal for strongly divisible instances.

THEOREM 1. *If (L, C) is strongly divisible, then an FFD packing is always optimal.*

Proof. By Fact 1, the gap remaining in a bin is always either zero or at least as large as the last (smallest) item placed in that bin, which is at least as large as any remaining unpacked item. Thus, either the bin is full or it has room for the next item to be packed, so FFD starts a new bin only in the event that all previous bins are completely full, which implies that the packing is optimal. ■

Remark. The above proof in fact shows that FFD produces "perfect packings," in which every bin except possibly the last is completely full. Indeed, from the proof it is easy to see that even the much simpler Next Fit Decreasing (NFD) algorithm produces perfect packings under these assumptions.

The proof of Theorem 1 uses Fact 1 and hence does not carry over to weakly divisible instances (L, C). However, the theorem itself does carry over to this less restrictive case.

THEOREM 2. *If (L, C) is weakly divisible, then an FFD packing is always optimal.*

Proof. We show that there always exists an optimal packing containing a bin packed identically to the first bin of the FFD packing. Since the deletion of the items in that bin does not change the FFD packing of the remaining items and reduces the size of the optimal packing by exactly one bin, the desired result follows immediately by induction on the number of bins in the FFD packing.

Let $L = I_1, I_2, \ldots, I_n$ be the ordered list of items used in constructing the FFD packing. B_1 contains item I_1 in the FFD packing. In any optimal packing, let B denote the corresponding bin containing I_1. Choose an optimal packing that maximizes the index k of the first item in L on which B_1 and B differ, with $k = \infty$ if the two bins are identical. Hence if $k = \infty$ we are done. Otherwise, we first observe that, by the definition of FFD, I_k must belong to B_1 and not B, rather than vice versa. Now, by Fact 2, either the sum of all the items in B with index larger than k is less than I_k or some subset of those items sums exactly to I_k. In either case, we can remove the corresponding items from B, replace them with I_k, and place them in the bin that had contained I_k, without changing the number of bins in the optimal packing. But this creates a new optimal packing with a larger value of k, contradicting our original choice of an optimal packing. Thus it must be the case that $k = \infty$, i.e., that there exists an optimal packing containing a bin packed identically to the first FFD bin. The theorem follows. ■

For strongly divisible instances, we can obtain optimal performance even without sorting the items beforehand. More specifically, in this case the on-line algorithm First Fit is also optimal.

THEOREM 3. *If (L, C) is strongly divisible, then an FF packing is always optimal.*

Proof. We argue by induction on the bin index j that any item packed by FF in any bin B_j must be larger than the sum of the gaps in all bins B_i, $1 \le i < j$. The theorem then follows by taking B_j to be the rightmost bin into which FF packs an item (i.e., j is the number of bins in the FF packing), since then the sum of all the items strictly exceeds $(j - 1)C$, which implies that an optimal packing must also use j bins.

Clearly the desired result holds for $j = 1$. Suppose that it holds for all $j < k$ and consider the case $j = k$. Let $g_1, g_2, \ldots, g_{k-1}$ denote the gaps in the bins $B_1, B_2, \ldots, B_{k-1}$ at the time some item I is packed into B_k. Let B_l be the rightmost bin to the left of B_k such that g_l is nonzero, and let I' denote a smallest item currently in B_l. By Fact 1, $g_l = rI'$ for some positive integer r. Since I did not fit in B_l, we know that $I > g_l = rI'$, and hence, by the definition of divisible sequence we have $I \ge (r + 1)I'$. By the induction hypothesis, $I' > \sum_{1 \le i < l} g_i$ holds. Hence,

$$\sum_{1 \le i < k} g_i = \sum_{1 \le i \le l} g_i < g_l + I' = (r + 1)I' \le I,$$

completing the induction step. The theorem follows. ■

It is easy to see that this result does not extend to weakly divisible instances. For example, suppose the bin capacity is $2^i - 1$, and there are m items each of the sizes $2^{i-1}, 2^{i-2}, \ldots, 1$. Then the optimal packing needs only m bins, but if the list L is in increasing order, FF uses approximately

$$m\left(\frac{1}{2^i - 1} + \frac{1}{2^{i-1} - 1} + \cdots + 1\right)$$

bins, which exceeds $1.6m$ for $i \ge 8$.

It is not difficult to extend the strong divisibility results of this section to problems of packing bins of varying sizes. In the model of Friesen and Langston (1986) there is a finite set of bin capacities $C_1 > C_2 > \cdots > C_k$ and unbounded sets of bins of each size. The objective is to pack the items so that the total size of the bins used (and hence the total wasted space) is minimized.

Consider the following algorithm under the assumption that (L_i, C_i) is strongly divisible for all $1 \leq i \leq k$, where L_i is the sublist of L containing just those items not exceeding C_i. First, pack items by FFD into bins of capacity C_1 until the remaining, unpacked items sum to less than C_1, or until all items are packed. If items remain and the largest exceeds C_2, the algorithm places the remaining items into a bin of size C_1 and then halts. Otherwise, the above procedure is applied recursively to the remaining items and bins of sizes $C_2, C_3, \ldots$. The arguments used earlier can be extended easily to a proof of the optimality of this algorithm. We leave to the interested reader the more complicated situation of multiple bin capacities and only weak divisibility.

3. Multiprocessor Scheduling

The multiprocessor scheduling problem is in a sense a dual to one-dimensional bin packing. We are given a fixed number m of bins with unlimited capacity, and asked to pack the items of L into these bins so as to minimize the makespan of the packing, i.e., the maximum bin level, where the *level* of a bin is the sum of the items it contains. (Observe that the concepts of strong and weak divisibility are meaningless here, since there is no specified bin capacity.) A standard heuristic for this problem is the $O(n \log n)$ *Lowest Fit Decreasing* (*LFD*) algorithm, which over all L and n has a worst-case ratio of 4/3, as shown by Graham (1969). LFD works by first sorting the items so that $I_1 \geq I_2 \geq \cdots \geq I_n$. It then places I_1 in bin B_1, and thereafter packs the items in order, placing I_k in a bin whose current level is minimum (if there is a tie, the lowest-indexed such bin is chosen). For the general case, more sophisticated heuristics can guarantee better worst-case ratios. However, when L has divisible item sizes, LFD guarantees optimal packings.

Theorem 4. *Given an integer m and a list L with divisible item sizes, LFD produces a packing of L into m bins that minimizes the maximum bin level.*

Proof. For simplicity assume that L already satisfies $I_1 \geq \cdots \geq I_n$, and suppose that the LFD packing $\mathbf{B} = B_1, \ldots, B_m$ does not have minimum makespan. Let $\mathbf{B}^0 = B_1^0, \ldots, B_m^0$ be the bins in a minimum makespan packing of L that agrees with $\mathbf{B}$ on the longest initial substring of L, and let I_t be the first item in L that is in different bins in $\mathbf{B}$ and $\mathbf{B}^0$ (such an item exists by assumption). We show below that $\mathbf{B}^0$ can be transformed into a minimum makespan packing $\mathbf{B}^1$ that agrees with B on item I_t and all its predecessors, a contradiction. The theorem follows.

Suppose that I_t is in bin B_k in the LFD packing and in bin B_j^0 in $\mathbf{B}^0$, with $j \neq k$. Let J be $B_k^0 \cap \{I_{t+1}, \ldots, I_n\}$. Note that by the ordering of L, no item in J exceeds I_t in size. If the items in J sum to I_t or more, then by Fact 2, some subset J' of them must sum to *exactly* I_t. We thus can obtain $\mathbf{B}^1$ from $\mathbf{B}^0$ by interchanging I_t and the items in J', without changing the sum of the items in either bin, and thus without affecting the makespan.

On the other hand, suppose the sum of the items in J is less than I_t. Then $\mathbf{B}^1$ is obtained from $\mathbf{B}^0$ by interchanging I_t with *all* the items in J. Bin B_j^0 does not have its sum increased, and so cannot have caused the makespan to increase. On the other hand, by the definition of LFD, the sum of the items in bin B_k^0 from the first $t-1$ items of L cannot have exceeded that for bin B_j^0. Therefore, the new level of B_k^1 does not exceed the original level of B_j^0, and B_k^1 cannot have caused the makespan to increase either.

Thus in either case $\mathbf{B}^1$ has minimum makespan and agrees with $\mathbf{B}$ on the locations of I_1 through I_t, the desired contradiction. ■

A problem related to makespan minimization is that of *maximizing* the minimum bin level, given m and L. For this problem in general, Deuermeyer *et al.* (1982) showed that LFD can produce minimum bin levels as small as 3/4 times the optimum, and they showed that this bound is tight. For divisible item sizes, however, LFD once again guarantees optimality.

THEOREM 5. *Given an integer m and a list L with divisible item sizes, LFD produces a packing of L into m bins which maximizes the minimum bin level.*

Proof. The proof mirrors that of the previous theorem, except that now $\mathbf{B}^0$ is a packing that has the maximum possible minimum bin level and agrees with the LFD packing on the longest possible initial substring of L. The indices t, j, and k are as before, as are the sets J and (if required) J'. It remains to be shown that the indicated interchanges do not yield a lower minimum bin level. This is clearly true when the sum of the items in J exceeds I_t, in which case our interchange of I_t with J' changes no bin levels.

On the other hand, if the sum of the items in J is less than I_t, then (1) the level of B_k^0 does not decrease in $\mathbf{B}^1$, and (2) although the level of B_j^0 may decrease, it cannot drop below the original level of B_k^0. Thus the minimum level does not decline, and the desired contradiction is once again derived. ■

4. MAXIMIZING THE NUMBER OF ITEMS PACKED

In this problem we are given both a number m of bins and a capacity C for each bin; the problem now is to pack *as many* items from L as possible

into the given bins. The literature concentrates on two approximation algorithms, the First-Fit Increasing algorithm introduced in Section 2 and the *Iterated First Fit Decreasing* (*IFFD*) algorithm. FFI runs in time $O(n \log n)$, applying FF to the sequence $I_1 \leq I_2 < \cdots \leq I_n$, with only the first m bins of the packing retained. FFI can on occasion produce packings that contain as few as $\frac{3}{4}$ of the optimal number of items, and Coffman *et al.* (1978) proved that this bound is tight. The second algorithm, IFFD, takes time $O(n^2 \log n)$. It first orders the items so that $I_1 \geq I_2 \geq \cdots \geq I_n$ and defines lists L_i, $0 \leq i \leq n$, where L_i is obtained from L by deleting the first (largest) i items. It then performs FF on the list L_0, L_1, etc., until a list is encountered that FF packs into m or fewer bins, and the resulting packing is output. IFFD produces packings with at least 7/8 times the optimal number of items, as shown by Coffman and Leung (1979).

We prove below that both algorithms produce optimal packings when (L, C) is strongly divisible, and that IFFD remains optimal even if (L, C) is only weakly divisible (although FFI does not). First, however, we observe the following.

FACT 3. *Suppose L, C, and m are such that there exists a packing of k items from L into m bins of capacity C. Then there is a packing of the truncated list L_{n-k} into those bins.*

Proof. Suppose the given packing contains $j > 0$ items from the set $\{I_1, \ldots, I_{n-k}\}$ of items not contained in L_{n-k}. Then it must omit j items from $\{I_{n-k+1}, \ldots, I_n\}$, all smaller than the first j items. Replacing each member of the first set of j items in the packing by a distinct member of the second set thus yields the legal packing of L_{n-k}. ■

THEOREM 6. *For any L, C, and m in which (L, C) is weakly divisible, IFFD produces a packing that maximizes the number of items packed.*

Proof. This follows directly from Fact 3 and Theorem 2, which show that FFD must pack L_{n-k} into m bins if there is *any* way of so packing them. ■

THEOREM 7. *For any L, C, and m with (L, C) strongly divisible, FFI produces a packing that maximizes the number of items packed.*

Proof. Suppose that the optimum number of items packed is k and consider the list L_{n-k} (as defined above). By Fact 3 we know that the items in this list can be packed into the given m bins. Note that these items are also the first k items in the list ordered by increasing size (or else are isomorphic to them, if there is a tie for the kth-smallest item). Thus, by Theorem 3, FF restricted to these first k items must pack them all into the m bins. Since continuing to pack the remainder of the list with FF cannot change the positions of these first k items, it follows that FFI must pack at least k items, and hence must produce an optimal packing. ■

If (L, C) is only weakly divisible, FFI need no longer produce optimal packings. Suppose for instance that m is of the form $3N$, $C = 3$, and L consists of $3N$ items of size 1 and $3N$ items of size 2. Then there is a way of packing all $6N$ items in m bins, but FFI packs only $5N$ of them, for a ratio of 5/6, only slightly better than the 3/4 worst-case bound that holds in general.

5. Maximizing the Number of "Full" Bins

Suppose we are given a list L of items and a capacity C, and are asked for a maximum number of sets, each of which has items of total size at least C. This dual to ordinary one-dimensional bin packing was proposed and analyzed by Assmann *et al.* (1984), who presented a sequence of approximation algorithms, each providing a better guarantee than its predecessor, but at the expense of a greater running time. Two are relevant here. The first is the natural variant on Next Fit (NF), in which items are placed in the last partially filled bin until it has level C or greater, and then a new bin is started. NF takes linear time and, in the worst case, never fills fewer than one-half the optimal number of bins. The second algorithm is called *Iterated Lowest Fit Decreasing* (*ILFD*) and works by repeatedly picking a number of bins m and applying the multiprocessor scheduling algorithm LFD to L with m bins. Binary search is used to find an m such that the LFD packing for m bins has a minimum level of C or more and the LFD packing for $m + 1$ bins has a minimum level less than C. This takes $O(n \log^2 n)$ time and provides a guarantee of 3/4.

It is not difficult to see that, even when (L, C) is strongly divisible, NF need not generate optimum packings. Consider a list consisting of $2N$ repetitions of the ordered pair (1, 2) when $C = 2$. Here the optimal packing would have $3N$ full bins, whereas NF yields but $2N$ (overly) full bins. Suppose, however, that we consider NFD, the $O(n \log n)$ algorithm that rearranges L into decreasing order before applying NF. We then have the following result.

THEOREM 8. *If (L, C) is strongly divisible, then NFD produces a packing with the maximum possible number of full bins.*

Proof. By the proof of Theorem 1, NFD coincides with FFD for such an instance, and produces a packing in which all bins, except possibly the last, have levels precisely C. Such a packing must be optimal. ■

Now let us consider the situation in which (L, C) is only required to be weakly divisible. In this case, NFD no longer guarantees optimality. For instance, if L consists of N items of size 2 and N of size 1 and $C = 3$, then the optimal number of full bins is N but NFD fills only $5N/6$. The more

time-consuming ILFD algorithm can still guarantee optimal solutions in this case, however.

THEOREM 9. *If (L, C) is weakly divisible, then ILFD produces a packing that has the maximum possible number of full bins.*

Proof. Suppose that the maximum possible number of full bins is m. Then for all $m' \leq m$, there is a packing of L into m' bins with minimum level C or more. Thus by Theorem 5, LFD yields a packing with minimum level C or more for any such m'. It follows that ILFD finds m in its binary search for a maximum m' that causes LFD to fill all bins to level C or more. ■

6. TWO-DIMENSIONAL BIN PACKING

We now consider two-dimensional (geometric) bin packing, beginning with the strip packing version described in Section 1, in which rectangles are to be packed into a fixed-width strip so as to minimize the height of the packing. Let W denote the width of the strip. Let L_h and L_w represent the one-dimensional lists of the heights and widths, respectively. We have various combinations of possibilities, depending on whether L_h has divisible item sizes and whether (L_w, W) is weakly divisible, strongly divisible, or neither. (The concepts of strong and weak divisibility do not apply to heights since there is no height capacity in this version of two-dimensional bin packing.)

The *First Fit Decreasing Height* (*FFDH*) algorithm packs the rectangles in order of nonincreasing height, placing them on "shelves" that run the width of the strip. The initial shelf is simply the bottom of the strip. The algorithm proceeds by placing each rectangle in turn as far left as possible on the first (lowest) shelf on which it fits. Whenever a rectangle does not fit on any of the existing shelves, it is placed left justified on a new shelf resting on the top of the first (tallest) rectangle on the previously highest shelf. For the unrestricted case, it was shown by Coffman *et al.* (1980) that asymptotically, as the optimal packing height divided by the maximum rectangle height tends to infinity, the height of the FFDH packing is at most 17/10 times the optimal height and that this bound is tight.

THEOREM 10. *If (L_w, W) is strongly divisible and L_h unconstrained, then* $\mathrm{FFDH}(L) < \mathrm{OPT}(L) + h_1$, *where h_1 denotes the height of the tallest rectangle in L.*

Proof. Consider any FFDH packing of such a set of items. Let $y_1 \geq y_2 \geq \cdots \geq y_k$ $(y_1 = h_1)$ denote the heights of the shelves in the FFDH packing, indexed in order of their creation, and for convenience let $y_{k+1} =$

0. For $1 \le i < j \le k+1$, let g_{ij} denote the gap (width to the right of the rightmost rectangle) remaining on the ith shelf at the time the jth shelf was created. Also, let g_{ii} denote the gap remaining on the ith shelf immediately after it is created, i.e., g_{ii} equals W minus the width of the first rectangle placed on shelf i. Then it is not hard to see that the amount of empty area remaining on the ith shelf in the final FFDH packing is at most

$$\sum_{i \le j \le k} g_{ij}(y_j - y_{j+1}).$$

Hence the total empty area E in the FFDH packing satisfies

$$E \le \sum_{1 \le i \le k} \left(\sum_{i \le j \le k} g_{ij}(y_j - y_{j+1}) \right).$$

This can be rewritten as

$$E \le \sum_{1 \le j \le k} \left((y_j - y_{j+1}) \sum_{1 \le i \le j} g_{ij} \right).$$

By the proof of Theorem 3, we have for $1 \le j \le k$ that

$$\sum_{1 \le i < j} g_{ij} < W - g_{jj}.$$

Hence,

$$E < W \sum_{1 \le j \le k} (y_j - y_{j+1}) = W(y_1 - y_{k+1}) = h_1 W.$$

Since $\mathrm{OPT}(L) \ge (W \cdot \mathrm{FFDH}(L) - E)/W$, the theorem follows. ∎

Note that this result says that FFDH is asymptotically optimal as $\mathrm{OPT}(L)/h_1$ tends to infinity.

If (L_w, W) is only weakly divisible, then the performance of FFDH is roughly comparable to its performance in the unconstrained case, even if L_h has divisible item sizes. To see this, we consider a class of examples generalizing those used in Section 2 to show that FF performs poorly for weakly divisible instances. Let $W = 2^i - 1$. For $1 \le j \le i$, we have $2^{j-1}m$ rectangles of width 2^{j-1} and height 2^{1-j}, which we call *type-j* rectangles. Note that we can pack the complete set of rectangles perfectly in total height m by packing m strips of height 1 and width W, each containing for every j, $1 \le j \le i$, 2^{j-1} type-j rectangles placed on top of one another. On the other hand, FFDH will operate much as FF did in the earlier example,

forming for each j, $1 \leq j \leq i$, $m2^{j-1}/(2^{i-j+1} - 1)$ shelves of height 2^{1-j} containing only type-j rectangles. The total height of those shelves for such a j is thus $m/(2^{i-j+1} - 1)$. Hence the total height of the FFDH packing is

$$m\left(\frac{1}{2^i - 1} + \frac{1}{2^{i-1} - 1} + \cdots + 1\right),$$

which exceeds $1.6m$ for $i \geq 8$.

If (L_w, W) is only weakly divisible but L_h has divisible item sizes, one might consider the variant of FFDH in which one divides all shelves into subshelves of the current height whenever the height of the rectangles being packed changes. This would allow rectangles to be placed on top of one another within the original shelves. Although this may yield some advantages in practice, we observe that L_h has divisible item sizes in the above examples, and this variant yields the same poor packings for them as does FFDH.

We now turn to the problem of packing rectangles into identical rectangular bins, say of height H and width W. We analyze below the algorithm *Hybrid First Fit* (*HFF*) of Chung *et al.* (1982), which first packs the rectangles into a strip of width W using *FFDH* and then packs the resulting shelves into bins of capacity H using FFD.

THEOREM 11. *If* (L_w, W) *is strongly divisible and* (L_h, H) *is weakly divisible, then* $\mathrm{HFF}(L) \leq \mathrm{OPT}(L) + 1$.

Proof. For any list L of rectangles ordered by nondecreasing height, let y_k denote the height of the first shelf to be placed in the bin HFF(L) by the HFF algorithm. Without loss of generality, we can assume that the rectangle that starts this shelf is the last rectangle in L, since the deletion of any rectangles that follow it would leave the HFF packing unchanged and cannot increase the number of bins in the optimal packing. Let H' denote the largest multiple of y_k that does not exceed the bin height H. By the fact that the item heights form a divisible sequence, and the fact that no item in L has a height less than y_k, we know immediately that the sum of the heights of the shelves in every bin of the HFF packing must be an integer multiple of y_k. Since the shelf of height y_k did not fit in any preceding bin, it follows that the heights of the shelves in each preceding bin must sum to exactly H'. By the proof of Theorem 10, the total area of all the rectangles in L is therefore at least

$$W((\mathrm{HFF}(L) - 1)H' + y_k) - y_1 W.$$

But the total area of the rectangles in any optimal bin cannot exceed $H'W$;

since the heights of all items in L are integer multiples of y_k, the sum of the heights of the rectangles in any vertical slice of an optimal bin is at most H'. Therefore, we must have

$$\begin{aligned}\mathrm{OPT}(L) &\geq (W((\mathrm{HFF}(L) - 1)H' + y_k) - y_1 W)/H'W \\ &\geq \mathrm{HFF}(L) - 1 + (y_k - y_1)/H'.\end{aligned}$$

Hence

$$\mathrm{HFF}(L) \leq \mathrm{OPT}(L) + 1 + (y_1 - y_k)/H',$$

and, since both $\mathrm{HFF}(L)$ and $\mathrm{OPT}(L)$ are integers and since $y_1 \leq H'$ and $y_k > 0$, we can conclude that $\mathrm{HFF}(L) \leq \mathrm{OPT}(L) + 1$, as desired. ■

Note the trivial observation that if (L_w, W) is only weakly divisible but (L_h, H) is strongly divisible, we can interchange height and width and obtain the same good results as in Theorem 11. If neither pair is strongly divisible, however, the performance of HFF need not be close to optimal, since the initial shelf packing phase need not be. If one pair, say (L_w, W), is strongly divisible but the other dimension is unrestricted, HFF cannot be close to optimal either, because we can mimic the 11/9 examples for FFD using rectangles of width W. Hence a question left open in this section is: Does HFF satisfy a bound close to 11/9 when (L_w, W) is strongly divisible and L_h is unconstrained?

7. Vector Packing

The vector packing problem investigated by Garey *et al.* (1976) is a multidimensional generalization of one-dimensional bin packing motivated by problems of scheduling under resource constraints, but without the geometric flavor of the problems discussed in the previous section. In the d-dimensional version, the "size" of an item is a vector $I = (I[1], I[2], \ldots, I[d])$ and the capacity of a bin is a vector $C = (C[1], C[2], \ldots, C[d])$. No bin is allowed to contain items whose vector sum exceeds C in any component. Note the distinction between two-dimensional vector packing and two-dimensional geometric packing. The former problem can be viewed as rectangle packing with the following constraints: The lowest rectangle in a bin is placed in the lower left corner; the lower lefthand vertex of any other rectangle in a bin coincides with the upper righthand vertex of the rectangle beneath it.

For vector packing, we say that (L, C) is weakly (strongly) divisible if and only if $(L^i, C[i])$ is weakly (strongly) divisible for each i, $1 \leq i \leq d$,

where L^i is the list of ith components of L. This is a case where divisibility restrictions do not significantly reduce problem complexity. In particular, we know of no way to find optimal solutions even for strongly divisible instances of dimension 2. Moreover, although the divisibility constraint is a handicap in proving complexity results, we can show that the problem of vector packing for strongly divisible instances remains NP-hard for fixed dimensions as small as 5. (See the Appendix.)

One can still ask, however, whether the strong divisibility constraint at least *improves* the worst-case performance guarantees of standard algorithms. Two approximation algorithms have received substantial study in the literature, First Fit and an adaptation of FFD, in which the items are ordered by decreasing values of $\max_{1 \le i \le d} I[i]/C[i]$. In general, neither algorithm is especially promising for high values of d; the asymptotic worst-case ratio for FF is $d + 0.7$ and that for FFD is not much better, having been shown to be at least $d + (d - 1)/(d(d + 1))$, although no more than $d + 1/3$. The strong divisibility constraint does help here, enabling us to prove a better upper bound for both FF and FFD with a much simpler proof. Unfortunately, the new bound cannot be viewed as a major improvement.

THEOREM 12. *For any dimension d, both FFD and FF have asymptotic worst-case ratios of precisely d when restricted to strongly divisible instances.*

Proof. We prove the theorem by establishing the following two claims:

(a) For all instances X, $\mathrm{FF}(X) \le d\ \mathrm{OPT}(X) + d$.

(b) There exist strongly divisible instances X of d-dimensional bin packing with $\mathrm{FFD}(X)/\mathrm{OPT}(X)$ arbitrarily close to d.

To establish (a), let $X = (L, C)$ be a strongly divisible instance. We may assume without loss of generality that all components of the capacity C are equal to 1. (If not, we could simply divide $C[i]$ and all ith components of item sizes by $C[i]$. The FF and FFD packings would not be affected.)

For each item I in L, let $c(I) = \sum_{i=1}^{d} I[i]$. Extend this measure to sets of items in the obvious way: $c(S) = \sum_{I \in S} c(I)$. We show below that $c(L) > \mathrm{FF}(X) - d$. Since clearly $c(L) \le d\ \mathrm{OPT}(X)$, (a) follows.

Let us examine the FF packing P. Let H be the set of bins B for which $c(B) \ge 1$ and let G be the remainder. For $B \in G$, let $c_i(B)$ be the sum of the ith component values for the items in B. Divide G into d groups depending on their maximum component value. To be precise, let the ith class G_i consist of all those bins $B \in G$ for which $c_i(B) > c_j(B)$, $1 \le j < i$, and $c_i(B) \ge c_j(B)$, $i < j \le d$. Let L_i be the set of items in bins of G_i.

We claim that the items of L_i are packed into the bins of G_i exactly as they would have been by FF had we considered only the one-dimensional bin packing problem consisting of L_i restricted to its ith components, ordered as they were packed in P. Suppose not. Then there must have been some item I in L_i that failed to go in a bin of G_i, even though it would not have caused the ith component of the bin's level to exceed the capacity constraint. Instead, it failed to go in because it would have violated the capacity constraint in some other component, say j. Since I eventually goes into a bin of G, we have $I[j] < 1$. Since the instance is strongly divisible, this implies $I[j] \leq 1/2$. Thus we must have had $c_j(B) > 1/2$. But then, since by definition of G_i, $c_i(B) \geq c_j(B)$, we have $c(B) \geq 1$, a contradiction of our definition of G.

Thus by the proof of Theorem 2, $c(G_i) \geq c_i(G_i) > |G_i| - 1$. But then we conclude that $c(L) > |H| + \sum_{i=1}^{d} (|G_i| - 1) = \mathrm{FF}(X) - d$, as claimed, thus proving (a).

To prove (b), we construct an infinite sequence of instances X_m, $m > d$, such that $\mathrm{FFD}(X_m) = md$ and $\mathrm{OPT}(X_m) \leq m + 1$. In instance X_m, $C[i] = 2^{2m+3d}$, $1 \leq i \leq d$, and there are d different types of items. An item of type T_i, $1 \leq i \leq d$, has an entry of 1 in each of its first $i - 1$ components, an entry of 2^{m+3d-i} in its ith component, and 0 in its last $d - i$ components. There are $m2^{m+i}$ such items.

By the action of FFD described above, these items are ordered so that all T_1-items come first, followed by the T_2-items, all T_3-items, etc., in order. The packing thus uses md bins, as each class T_i in turn fills up m bins, 2^{m+i} items to a bin, with no room left in the ith component for any of the later items, all of which have a nonzero ith component.

To pack the items into $m + 1$ bins, we place $2^{m+i} - 1$ items of type T_i in each of the first m bins, $1 \leq i \leq d$. These all fit together since the total for the ith component is bounded by

$$(C[i] - 2^{m+3d-i}) + \sum_{j>i} 2^{m+j} < C[i] - 2^{m+2d} + 2^{m+d+1} \leq C[i]$$

There remain m items of each type, but these all fit together by the above argument since $m < 2^{m+i}$ for all i, $1 \leq i \leq d$. Thus we have established (b) and the theorem follows. ■

If we require only that the instance be weakly divisible, it is not difficult to show that the asymptotic worst-case ratio for FF increases to at least $d + 0.6$, i.e, $d - 1$ plus the worst-case ratio for FF and weakly divisible instances of one-dimensional bin packing. The asymptotic worst-case ratio for FFD in this situation remains open. However, the techniques used to prove the general lower bound $(d - 1)/(d(d + 1))$ do not seem to

survive the weak divisibility constraint, which suggests that the worst-case ratio of d might continue to hold in this case.

8. Dynamic Bin Packing

Dynamic bin packing extends the standard one-dimensional model by introducing arrivals and departures of items. It thus applies, for example, to dynamic allocation in computer storage devices that are partitioned into units (bins), e.g., disk cylinders or drum sectors, where the overlapping of a stored item from one unit to the next is not allowed.

In this case the FF algorithm treats the bins as being divided into two lists, the empty bins and the occupied bins, with the latter list maintained in nondecreasing order according to the last time each bin became nonempty. When an item arrives, the occupied-bin list is scanned, and the new item is placed in the first nonempty bin that has sufficient available capacity. If the item does not fit in any of the nonempty bins, then the item is placed in a new empty bin, which is appended to the list of nonempty bins. Departing items simply free up the space they have been occupying; whenever a bin becomes empty, it is removed from the list of nonempty bins and placed on the empty-bin list.

Let L be an arbitrary sequence of arrivals and departures; the lengths of inter-event intervals are irrelevant here and need not be specified. During the processing of L, let $\mathrm{FF}(L)$ and $\mathrm{OPT}(L)$ denote the respective maximum number of nonempty bins ever needed by FF and an optimal packing for storing the currently active items, i.e., those that have arrived but have not yet departed. We assume that optimal packings are unconstrained, in that the currently active items can at any time be repacked to make room for new items. Thus FF is handicapped with respect to an optimal algorithm in two ways: it is on-line and it cannot rearrange. By themselves these two restrictions severely limit how good an algorithm can be. It was shown by Coffman *et al.* (1983) that no algorithm obeying these restrictions could have an asymptotic worst-case ratio better than 2.5. In the particular case of FF, they showed that asymptotically, as $\mathrm{OPT}(L)$ becomes large, the ratio $\mathrm{FF}(L)/\mathrm{OPT}(L)$ over all L is bounded by a constant which is at least 2.770 and at most 2.898.

Restricting attention to strongly divisible instances improves matters somewhat, but asymptotic optimality is still well beyond the hope of any on-line, nonrearranging algorithm. As we shall show, any such algorithm must still have an asymptotic worst-case ratio of 2.384. Now, however, the gap between the general lower bound and that for FF disappears; FF attains the above bound and hence can be viewed as the *best* possible on-

line algorithm under the restriction to strongly divisible instances. More precisely, we have the following result.

THEOREM 13. *If (L, C) is strongly divisible, and the sizes of items in L form the sequence $s_1 > s_2 > \cdots > s_k$, $k \geq 2$, then the dynamic version of FF satisfies*

$$\mathrm{FF}(L) \leq (\mathrm{OPT}(L) + 1) \prod_{i=1}^{k-1} \left(1 + \frac{s_i}{C} - \frac{s_{i+1}}{C}\right).$$

Moreover, for any given $s_1 > s_2 > \cdots > s_k$, $k \geq 2$, and any on-line, nonrearranging algorithm A, there exist lists L with these item sizes and with arbitrarily large values of $\mathrm{OPT}(L)$ *such that*

$$\mathrm{A}(L) \leq \mathrm{OPT}(L) \prod_{i=1}^{k-1} \left(1 + \frac{s_i}{C} - \frac{s_{i+1}}{C}\right).$$

The above claims can be derived from this theorem as follows. By a straightforward inductive argument, which begins by setting $s_k = 0$ and then repeatedly optimizes the two smallest yet-undetermined sizes, we can show that the product

$$\prod_{i=1}^{k-1} \left(1 + \frac{s_i}{C} - \frac{s_{i+1}}{C}\right)$$

is maximized when $s_i = C/2^{i-1}$, $1 \leq i \leq k-1$, and s_k is chosen arbitrarily small. With these values, the product becomes

$$\left(1 + \frac{1}{2^{k-2}}\right) \prod_{i=1}^{k-2} \left(1 + \frac{1}{2^i}\right).$$

As k tends to infinity the latter product approaches a limiting value of 2.384. . . . Hence, by Theorem 13, this constant is the asymptotic worst-case ratio for FF and a lower bound on the asymptotic worst-case ratio for *any* on-line, nonrearranging algorithm.

Proof of Theorem 13. First note that, under the assumption that (L, C) is strongly divisible, it follows from our analysis for ordinary FFD packing that $\mathrm{OPT}(L)$ is simply the ceiling of (1/C) times the maximum sum of currently active items ever encountered during the processing of L. Consider the occupied-bin list at an arbitrary point during the processing of L. Suppose an item of size s_k was just packed in some bin B_j. Then, by Fact 1 and the definition of FF, bins $B_1, B_2, \ldots, B_{j-1}$ must all be

completely full. Hence, by definition of OPT(L), we must have $j \leq$ OPT(L), i.e., an item of size s_k can never be packed beyond bin $B_{\text{OPT}(L)}$. For later convenience, let $j_0 = \text{OPT}(L)$.

Next, consider the case in which an item of size s_{k-1} was just packed in some bin $B_j, j > j_0$. By Fact 1 and the definition of FF, each of the bins B_1, $B_2, \ldots, B_{j_0}$ must be filled to a level of at least $C - s_{k-1} + s_k$. Since no item of size s_k can appear in the bins $B_{j_0+1}, \ldots, B_{j-1}$, these bins must all be completely full. Hence we have

$$C \cdot \text{OPT}(L) \geq j_0(C - s_{k-1} + s_k) + C(j - 1 - j_0) + s_{k-1}.$$

Substituting $j_0 = \text{OPT}(L)$ and simplifying, we obtain

$$j < (\text{OPT}(L) + 1)\left(1 + \frac{s_{k-1}}{C} - \frac{s_k}{C}\right).$$

Letting j_1 equal the right-hand side of this inequality, we thus have that no item of size s_{k-1} can never be packed in a bin B_j for which $j > j_1$.

We continue this argument inductively, defining a sequence $j_0 \leq j_1 \leq \cdots \leq j_{k-1}$ such that each j_i has the property that no item of size s_{k-i} can ever be packed in a bin B_j having $j > j_i$. We claim that the values

$$j_i = \text{OPT}(L) + 1 + \sum_{l=0}^{i-1} j_l \left(\frac{s_{k-l-1}}{C} - \frac{s_{k-l}}{C}\right), \qquad i \geq 1$$

suffice for this.

It is easy to see for $i = 1$ that this is equivalent to what we have already shown. So suppose the claim holds for $1, 2, \ldots, i-1$, and consider a point at which an item of size s_{k-i} has just been packed into a bin $B_j, j > j_{i-1}$. The level of each preceding bin must be at least $C - s_{k-i} + q$, where q is the smallest item size that can appear in the bin, as determined by the bin index relative to the already determined values of $j_0, j_1, \ldots, j_{i-1}$. From this we obtain the recurrence

$$\begin{aligned} C \cdot \text{OPT}(L) > {} & j_0(C - s_{k-i} + s_k) \\ & + (j_1 - j_0)(C - s_{k-i} + s_{k-1}) \\ & + (j_2 - j_1)(C - s_{k-i} + s_{k-2}) \\ & + \cdots \\ & + ((j-1) - j_{i-1})(C - s_{k-i} + s_{k-i}) \end{aligned}$$

or, rearranging and simplifying,

$$j < \mathrm{OPT}(L) + 1 + \sum_{l=0}^{i-1} j_l \left(\frac{s_{k-l-1}}{C} - \frac{s_{k-l}}{C} \right) = j_i, \qquad i \geq 1,$$

as claimed.

We then observe that, for $i > 1$,

$$\frac{j_i}{j_{i-1}} = 1 + \frac{s_{k-i}}{C} - \frac{s_{k-i+1}}{C},$$

from which it follows that an equivalent way of expressing j_i is

$$j_i = (\mathrm{OPT}(L) + 1) \prod_{l=k-i}^{k-1} \left(1 + \frac{s_l}{C} - \frac{s_{l+1}}{C} \right), \qquad i \geq 1.$$

Since $\mathrm{FF}(L) \leq j_{k-1}$, the upper bound of the theorem is proved.

For any on-line, nonrearranging algorithm A (including FF), it is not difficult to construct lists L that show that the multiplicative constant in this upper bound cannot be beaten. The following steps illustrate how to specify a suitable sequence of arrivals and departures, given A. Let $\mathrm{OPT}(L)$ be any integer such that

$$m_i = \mathrm{OPT}(L) \prod_{l=k-i}^{k-1} \left(1 + \frac{s_l}{C} - \frac{s_{l+1}}{C} \right)$$

is an integer for all $i = 1, 2, \ldots, k-1$, and set $m_0 = \mathrm{OPT}(L)$.

1. We begin by having $m_0 C/s_k$ items of size s_k arrive. Note that A must use at least m_0 bins to house them. Let $B_1, B_2, \ldots, B_{m_0}$ be the first m_0 bins that receive such items.

2. Next, all but a single item of size s_k from each of these specified m_0 bins are made to depart, and the total amount removed is then reformed into items of size s_{k-1}, which are made to arrive. Some of these may go into the first m_0 bins, but can only fill them up to level $C - s_{k-1} + s_k$, and so at least $m_0(s_{k-1} - s_k)/C = m_1 - m_0$ additional bins must receive items of size s_{k-1}, even if all are filled to level C. (At this point we have the desired lower bound for $k = 2$.) Let the first $m_1 - m_0$ of these additional bins be denoted B_{m_0+1} through B_{m_1}.

3. Next, everything but a single item of size s_k is made to depart from each bin B_i, $l \leq i \leq m_0$, everything but a single item of size s_{k-1} is made to depart from each bin B_i, $m_0 < i \leq m_1$, and any remaining bins are *completely* emptied out. The total amount removed is then reformed into items of size s_{k-2}, which are made to arrive. These can fill each of the first

m_0 bins to level at most $C - s_{k-2} + s_k$ and each of the next $m_1 - m_0$ bins to level at most $C - s_{k-2} + s_{k-1}$. Thus each of these bins has a shortfall of at least $s_{k-2} - s_{k-1}$ from its maximum allowed level in the previous packing, and so at least $m_2 - m_1$ additional bins must be used, even if all are filled to level C. (At this point we have the lower bound for $k = 3$).

This process continues in the obvious manner, repeatedly causing all but a smallest item to depart from each of the first m_j occupied bins (where s_{k-j} was the size of the last item packed), and then reforming the total removed into items of the next larger size (s_{k-j-1}), which arrive next. It is straightforward to check that, when packing the items of this size, the occupied-bin list will be extended to contain at least m_{j+1} bins, from which the lower bound follows. ■

APPENDIX

This appendix is devoted to the proof of the following result, cited in Section 7. For the technical background on NP-completeness, see Garey and Johnson (1979).

THEOREM. *Given a strongly divisible instance (L, C) of five-dimensional vector packing and an integer bound B, the problem of determining whether the items of L can be packed in B bins of capacity C is NP-complete.*

Proof. The proof is via a transformation from 3-DIMENSIONAL MATCHING (3DM). Recall that an instance of 3DM consists of three equal-sized sets X, Y, and Z, together with a collection $S = \{s_1, \ldots, s_n\}$ of three-element sets, each set $s \in S$ containing one element each from X, Y, and Z. The question asked is whether S contains a *matching*, i.e., a subcollection $M \subseteq S$ such that $|M| = |X| = |Y| = |Z|$ and $\cup_{s \in M} s = X \cup Y \cup Z$.

We show how, given an instance (X, Y, Z, S) of 3DM one can in polynomial time construct a corresponding instance (L, C, B) of five-dimensional vector packing, such that (X, Y, Z, S) has a matching if and only if L has a packing in B bins of capacity C. Let $X = \{x_i : 0 \le i < m\}$, $Y = \{y_i : 0 \le i < m\}$, and $Z = \{z_i : 0 \le i < m\}$. We may assume without loss of generality that m is 2^H for some integer H and that $3m \le 2^m$.

We begin our construction by setting $C = (2^m, 2^m, 2^m, 2^m, 2^m)$ and $B = |S| - m + 1$. We have three classes of items. First, for each set $s = \{x_i, y_j, z_k\}$ in S we have an item $I_s = (2^i, 2^j, 2^k, 2^{m-H}, 0)$. Second, we have a special item $I_0 = (1, 1, 1, 0, 2^m)$. Note that a bin which contains I_0 and is

completely full in the first three components can be constructed from these items if S contains a matching.

The third class of items is chosen so that the remaining $|S| - m$ bins are also full in their first three components. There are four types of these "garbage collection" items. First, there are $|S| - m$ identical items of size $(1, 1, 1, 0, 1)$. To simplify the definitions of the remaining three types of garbage collection items, define $n(u)$ to be the number of sets in S containing u, for each $u \in X \cup Y \cup Z$. For each $x_i \in X$ we then have $n(x_i) - 1$ items of size $(2^h, 0, 0, 0, 1)$ for each $h \neq i$, $0 \leq h < m$. Similarly, for each $y_i \in Y$ we have $n(y_i) - 1$ items of size $(0, 2^h, 0, 0, 1)$ for each $h \neq j$, and for each $z_k \in Z$ we have $n(z_k) - 1$ items of size $(0, 0, 2^h, 0, 1)$ for each $h \neq k$. This completes the construction. It clearly can be performed in polynomial time, so to complete the proof we need only show that the constructed set of items can be packed into B bins of capacity C if and only if the original 3DM instance had a matching.

First note that for each of the first three components, the sum of the projections of the item sizes on that component exactly equals $(|S| - m + 1)2^m = BC_i$. Thus, as claimed, if there is a packing of L in B bins, each bin must be completely full in each of its first three components.

Suppose now that there exists a matching M. The packing of L into $B = |S| - m + 1$ bins proceeds as follows. In the first bin we place I_0 together with all the I_s, $s \in M$. As remarked before, these items must fit and completely fill up the first three components. The remaining $|S| - m$ items I_s, s not in M, go one per bin, as do the $|S| - m$ items of size $(1, 1, 1, 0, 1)$. The bin containing $\{x_i, y_j, z_k\}$ is then completed by adding $3(m - 1)$ garbage collection items: $(2^h, 0, 0, 0, 1)$, $h \neq i$, $(0, 2^h, 0, 0, 1)$, $h \neq j$, and $(0, 0, 2^h, 0, 1)$, $h \neq k$. It is straightforward to verify that this packing accounts for all items and does not overfill any bin. (The capacity constraint is not violated in the fifth component because of our assumption that $3m \leq 2^m$.)

Conversely, suppose there is a packing into $|S| - m + 1$ bins. Consider the bin containing I_0. By the above remarks, we know that this bin must be completely full in its first three components. Since it contains I_0, which completely fills the fifth component by itself, it can contain no garbage collection items, all of which have a fifth-component value of 1. Thus, the only items it can contain besides I_0 are I_s items. It can contain no more than m such items, since each has a fourth component equal to C_4/m. It must contain at least m of them, since the sum of their first components (as well as their second and third components) must be precisely $2^m - 1$. Thus there must be exactly m, and each value 2^i, $0 \leq i < m$, must be represented in each component. Consequently, this set of m items I_s must be a matching.

The vector packing instance thus has a solution if and only if the 3DM instance has a matching, and the theorem is proved. ■

REFERENCES

ASSMANN, S. J., JOHNSON, D. S., KLEITMAN, D. J., AND LEUNG, J. Y.-T. (1984), On a dual version of the one-dimensional bin packing problem, *J. Algorithms* **5,** 502–525.

CHUNG, F. R. K., GAREY, M. R., AND JOHNSON, D. S. (1982), On packing two-dimensional bins, *SIAM J. Algebraic Discrete Methods* **3,** 66–76.

COFFMAN, E. G., JR., GAREY, M. R., AND JOHNSON, D. S. (1983), Dynamic bin packing, *SIAM J. Comput.* **12,** 227–258.

COFFMAN, E. G., JR., GAREY, M. R., AND JOHNSON, D. S. (1984), Approximation algorithms for bin packing—An updated survey, *in* "Algorithm Design for Computer System Design" (G. Ausiello, M. Lucertini, and P. Serafini, Eds.), pp. 49–106, Springer-Verlag, New York.

COFFMAN, E. G., JR., GAREY, M. R., JOHNSON, D. S., AND LAPAUGH, A. S. (1985), Scheduling file transfers, *SIAM J. Comput.* **14,** 744–780.

COFFMAN, E. G., JR., GAREY, M. R., JOHNSON, D. S., AND TARJAN, R. E. (1980), Performance bounds for level-oriented two-dimensional packing algorithms, *SIAM J. Comput.* **9,** 808–826.

COFFMAN, E. G., JR., AND LEUNG, J. Y.-T. (1979), Combinatorial analysis of an efficient algorithm for processor and storage allocation, *SIAM J. Comput.* **8,** 202–217.

COFFMAN, E. G., JR., LEUNG, J. Y.-T., AND TING, D. W. (1978), Bin packing: Maximizing the number of pieces packed, *Acta Inform.* **9,** 263–271.

DEUERMEYER, B. L., FRIESEN, D. K., AND LANGSTON, M. A. (1982), Maximizing the minimum processor finish time in a multiprocessor system, *SIAM J. Algebraic Discrete Methods* 190–196.

FRIESEN, D. K., AND LANGSTON, M. A. (1986), Variable sized bin packing, *SIAM J. Comput.* **15,** 222–230.

GAREY, M. R., GRAHAM, R. L., JOHNSON, D. S., AND YAO, A. C. (1976), Resource constrained scheduling as generalized bin packing, *J. Combin. Theory Ser. A* **21,** 257–298.

GAREY, M. R., AND JOHNSON, D. S. (1979), "Computers and Intractability: A Guide to the Theory of NP-Completeness," Freeman, San Francisco.

GRAHAM, R. L. (1969), Bounds on multiprocessor timing anomalies, *SIAM J. Appl. Math.* **17,** 263–269.

HIRSCHBERG, D. S. (1973), A class of dynamic memory allocation algorithms, Comm. ACM **16,** 615–618.

JOHNSON, D. S., DEMERS, A., ULLMAN, J. D., GAREY, M. R., AND GRAHAM, R. L. (1974), Worst-case performance bounds for simple one-dimensional packing algorithms, *SIAM J. Comput.* **3,** 299–325.

KNUTH, D. E. (1973), "Fundamental Algorithms," Vol. 1, 2nd ed., Addison–Wesley, Reading, MA.

ULLMAN, J. D. (1975), Complexity of scheduling problems, *in* "Computer and Job-Shop Scheduling Theory" (E. G. Coffman, Jr., Ed.), pp. 139–164, Wiley, New York.